Regex: The Complete Beginner's Guide

You do not need to know programming to start. Every idea is explained in simple words first, and then shown with an example. All code examples were run and tested, so the results you see are real.

Table of Contents

1. What is Regex? · p.1
2. Why do we use Regex? · p.2
3. Where is Regex used? · p.3
4. How does Regex work? · p.4
5. Lesson 1: Plain text and special characters · p.5
6. Lesson 2: The dot · p.6
7. Lesson 3: Character classes · p.6
8. Lesson 4: Quantifiers (how many times) · p.7
9. Lesson 5: Anchors (where in the text) · p.7
10. Lesson 6: Groups · p.8
11. Lesson 7: OR (alternation) · p.9
12. Lesson 8: Lookarounds (peeking) · p.10
13. Lesson 9: Flags (settings) · p.10
14. Lesson 10: Regex in Python · p.11
15. Lesson 11: Real-life examples · p.12
16. Lesson 12: How to build a pattern step by step · p.14
17. Common mistakes · p.15
18. Big Project: Cleaning messy data · p.16
19. Project structure: which file does what · p.17
20. Where does raw data come from? · p.19
21. Practice exercises · p.20
22. Cheat sheet · p.21
23. Small dictionary of words · p.22
24. What to do next · p.23

1. What is Regex?

Regex is short for **Regular Expression**.

A regex is a **pattern** that describes the *shape* of some text. A computer then uses that pattern to find matching text.

A simple way to think about it

Imagine you tell a friend: *"Find every phone number in this big book."*

You cannot say the exact numbers, because you do not know them. But you know the **shape**: three digits, a dash, three digits, a dash, four digits.

Regex lets you write that shape for the computer:

```
\d{3}-\d{3}-\d{4}
```

- `\d` means "a digit"
- `{3}` means "exactly 3 times"
- `-` means "a dash"

So this pattern finds `555-123-4567`, `800-555-0199`, and any other number with that shape.

Normal search vs regex search

Normal search (Ctrl+F)	Regex search
Finds one exact word, like `cat`	Finds a **shape**, like "any 4-digit year"
Cannot find "any email"	Can find every email in a file
Cannot fix or clean text	Can find and replace thousands of items at once

---

2. Why do we use Regex?

Reason	Simple explanation	Example
Find patterns	Search for shapes, not fixed words	Find every date in a document
Check input	Make sure typed data is in the right format	Is this a real-looking email?
Take out data	Pull small pieces out of big messy text	Get all IP addresses from a log

Reason	Simple explanation	Example
Clean data	Fix spaces, capital letters, symbols	`john SMITH` becomes `John Smith`
Replace in bulk	Change thousands of items in one step	Change all `10/05/2026` into `2026-10-05`
Save time	One short pattern replaces many lines of code	Real programmers use it every day
Works everywhere	Python, JavaScript, Java, PHP, Excel tools, code editors	Learn once, use anywhere

---

3. Where is Regex used?

Area	How regex helps
Code editors (VS Code, Notepad++)	Search and replace across many files
Websites and apps	Check that an email, phone number, or password has the correct format
Server logs	Find errors, IP addresses, and times in huge log files
Data cleaning	Fix messy spreadsheets and text files before analysis
Web scraping	Pull prices, links, and titles from web pages
Web development	Match web addresses like `/user/123` to the right page
Security	Spam filters, firewalls, spotting suspicious activity in logs
AI and language processing	Prepare and split text for machine learning
Command line	Tools like `grep` and `sed` use regex

In one line: regex is used whenever you need to **find, check, take out, or change** patterns in text.

---

4. How does Regex work?

The computer program that reads your pattern is called a **regex engine**. It works in five simple steps:

1. **Read** your pattern and understand it.
2. **Start** at the first letter of the text.
3. **Try** to match the pattern from that spot.
4. If it fails, **move one letter forward** and try again.
5. **Report** the match when found, or say "no match" at the end.

Example: pattern `c.t` on the text `my cat sat`

Position	Question the engine asks	Answer
`m`	Is this `c`?	No, move on
`y`	Is this `c`?	No
space	Is this `c`?	No
`c`	Is this `c`?	**Yes**, check the next part
`a`	`.` means any letter, so is `a` fine?	**Yes**
`t`	Is this `t`?	**Yes. Match found:** `cat`

Backtracking (going back and trying again)

Sometimes the engine takes too much and has to **give some back**.

Pattern `a.*b` on the text `axxbxx`:

1. `a` matches the first `a`.
2. `.*` is greedy, so it grabs everything left: `xxbxx`.
3. Now `b` needs a letter, but nothing is left. **Fail.**
4. The engine **gives back** letters one at a time until `b` fits.
5. Final match: `axxb`.

This "give back and try again" is called **backtracking**.

Two kinds of engines

Kind	How it works	Used by
Backtracking	Tries one path, goes back on failure. Has more features.	Python, JavaScript, Java, PHP
Automaton (DFA)	Follows all paths at once. Always fast, fewer features.	`grep`, Google's RE2, Rust

---

5. Lesson 1: Plain text and special characters

The simplest regex is just the word itself.

Pattern	Text	Matches
`cat`	`cat concat Cat`	`cat`, `cat`

Why two matches? The second `cat` is hiding inside `concat`. The capital `Cat` is **not** matched, because regex cares about capital letters (until you change a setting, see Lesson 9).

Special characters

Some characters have a **special job** in regex:

```
. ^ $ * + ? { } [ ] \ | ( )
```

If you want one of them to mean **itself**, put a backslash `\` before it. This is called **escaping**.

Pattern	Text	Matches
`3\.14`	`3.14 and 3x14`	`3.14` only

Without the backslash, `3.14` would also match `3x14`, because a plain `.` means "any character" (next lesson).

---

6. Lesson 2: The dot

The dot `.` means **any one character** (except a new line).

Pattern	Text	Matches
`c.t`	`cat cot c9t ct`	`cat`, `cot`, `c9t`

`ct` is not matched, because the dot needs **exactly one** character between `c` and `t`.

---

7. Lesson 3: Character classes

A **character class** is a set of allowed characters, written inside square brackets `[ ]`. It matches **one** character from that set.

Pattern	Text	Matches	Meaning
`[aeiou]`	`regex`	`e`, `e`	any vowel
`[a-c]+`	`abcdxyz cab`	`abc`, `cab`	letters from a to c
`[^0-9]+`	`ab12cd`	`ab`, `cd`	anything that is **not** a digit

The `^` right after `[` means **NOT**.

Shortcuts you will use all the time

Shortcut	Means	Example	Result
`\d`	a digit (0 to 9)	`\d+` on `Order 66 costs 120`	`66`, `120`
`\w`	a letter, digit, or underscore	`\w+` on `hi_there, you!`	`hi_there`, `you`
`\s`	a space, tab, or new line	`\s+` on `a b` (two spaces)	the two spaces
`\D`	**not** a digit		
`\W`	**not** a word character		
`\S`	**not** a space		

Memory trick: the capital letter is always the **opposite** of the small letter.

8. Lesson 4: Quantifiers (how many times)

A **quantifier** says how many times the thing before it can repeat.

Symbol	Meaning	Pattern	Text	Matches
`*`	zero or more times	`ab*c`	`ac abc abbbc`	`ac`, `abc`, `abbbc`
`+`	one or more times	`ab+c`	`ac abc abbbc`	`abc`, `abbbc`
`?`	zero or one time (optional)	`colou?r`	`color colour`	`color`, `colour`
`{3}`	exactly 3 times	`\d{3}`	`12 123 1234`	`123`, `123`
`{2,4}`	between 2 and 4 times	`\d{2,4}`	`1 12 123456`	`12`, `1234`, `56`
`{2,}`	2 or more times	`\d{2,}`	`1 12 123`	`12`, `123`

Look at the `{2,4}` row. The number `123456` was split into `1234` and `56`. The engine takes up to 4 digits first, then starts a new match.

Greedy vs lazy

By default, quantifiers are **greedy**: they take **as much as possible**.

Adding `?` after a quantifier makes it **lazy**: it takes **as little as possible**.

Text: `<b>bold</b><i>it</i>`

Pattern	Type	Matches
`<.+>`	greedy	`<b>bold</b><i>it</i>` (everything in one piece)
`<.+?>`	lazy	`<b>`, `</b>`, `<i>`, `</i>` (each tag separately)

Easy way to remember: greedy = eats the whole plate. Lazy = takes one bite.

9. Lesson 5: Anchors (where in the text)

Anchors do not match letters. They match a **position**.

Anchor	Meaning	Pattern	Text	Matches
`^`	start of the text	`^\w+`	`hello world`	`hello`
`$`	end of the text	`\w+$`	`hello world`	`world`
`\b`	edge of a word	`\bcat\b`	`cat concatenate cat.`	`cat`, `cat`

`\b` (word boundary) stops `cat` from matching inside `concatenate`.

Many lines

Normally `^` means the start of the **whole text**. With the multiline setting (`re.M`), it means the start of **each line**:

```
re.findall(r"^\w+", "one\ntwo\nthree", re.M)    # ['one', 'two', 'three']
```

---

10. Lesson 6: Groups

Brackets `( )` create a **group**. Groups do two jobs: they **keep together** part of a pattern, and they **save** the matched text so you can use it.

6.1 Capturing groups (save the matched parts)

```
m = re.search(r"(\d{4})-(\d{2})-(\d{2})", "Date 2026-10-10")
m.group(0)    # '2026-10-10'           the whole match
m.groups()    # ('2026', '10', '10')   the saved parts: year, month, day
```

6.2 Named groups (easier to read)

Give each group a name, so you do not have to count.

```
m = re.search(r"(?P<y>\d{4})-(?P<m>\d{2})", "2026-10")
m["y"]          # '2026'
m.groupdict()   # {'y': '2026', 'm': '10'}
```

6.3 Non-capturing groups `(?: )`

Use these when you only want to **group**, not save.

Pattern	Text	Matches
`(?:ab)+`	`ababab ab`	`ababab` , `ab`

The group lets `+` repeat the **whole** `ab` . Without it, `ab+` would repeat only the `b` .

6.4 Backreference `\1` (reuse what was saved)

`\1` means "the same text that group 1 matched".

Find repeated words:

Pattern	Text	Result
`\b(\w+)\s+\1\b`	`this is the the test`	`the`

Use saved groups when replacing:

```
re.sub(r"(\w+)@(\w+)", r"\2 at \1", "user@site")    # 'site at user'
```

---

11. Lesson 7: OR (alternation)

The pipe symbol `|` means **OR**.

```
cat|dog          matches "cat" or "dog"        (text: cat bird dog  ->  cat,
dog)
gr(?:a|e)y       matches "gray" or "grey"      (text: gray grey groy ->  gray,
grey)
```

Put the OR inside a group to limit how far it reaches.

---

12. Lesson 8: Lookarounds (peeking)

A **lookaround** lets the engine **peek** at the text next to the match, without including it in the result.

Type	Meaning	Pattern	Text	Matches
Lookahead `(?=...)`	followed by	`\d+(?=%)`	`50% 20 30%`	`50`, `30`
Lookbehind `(?<=...)`	preceded by	`(?<=\$)\d+`	`$40 and 50 and $60`	`40`, `60`
Negative lookbehind `(?<!...)`	**not** preceded by	`(?<!\$)\b\d+`	`$40 and 50 and $60`	`50`

Negative lookahead `(?!...)` means "**not** followed by". For example, `\d+(?!%|\d)` on `50% 20 30%` finds only `20`.

The `%` and `$` are checked, but not included in the match. That is the difference between a lookaround and a normal group.

---

13. Lesson 9: Flags (settings)

Flags change how the whole pattern behaves.

Flag (Python)	What it does	Example
`re.I`	ignore capital/small letters	`hello` finds `Hello`, `HELLO`, `hello`
`re.M`	`^` and `$` work on every line	see Lesson 5
`re.S`	the dot also matches a new line	`a.b` on `"a\nb"`: matches only with `re.S`
`re.X`	verbose mode: allows spaces and comments in long patterns	makes big patterns readable

---

14. Lesson 10: Regex in Python

```
import re
s = "Order #1234 shipped on 2026-10-10"
```

Function	What it does	Code	Result
`search`	first match **anywhere**	`re.search(r"\d+", s).group()`	`'1234'`
`match`	match only at the **start**	`re.match(r"\d+", s)`	`None` (text starts with "Order")
`fullmatch`	the **whole** text must match	`re.fullmatch(r"\d+", "123")`	a match
		`re.fullmatch(r"\d+", "123a")`	`None`
`findall`	list of **all** matches	`re.findall(r"\d+", s)`	`['1234', '2026', '10', '10']`
`finditer`	all matches **with positions**	`[(m.group(), m.start()) for m in re.finditer(r"\d+", s)]`	`[('1234', 7), ('2026', 23), ('10', 28), ('10', 31)]`
`sub`	**replace** matches	`re.sub(r"\d", "#", s)`	`Order ##### shipped on ####-##-##`
`sub` + function	replace using a calculation	`re.sub(r"\d+", lambda m: str(int(m.group())*2), "a1 b20")`	`a2 b40`
`split`	**cut** text at matches	`re.split(r"[,;\s]+", "a, b;c d")`	`['a', 'b', 'c', 'd']`
`escape`	make normal text safe to use in a pattern	`re.escape("a.b*c")`	`a\.b\*c`
`compile`	prepare a pattern once, reuse it	`p = re.compile(r"\d+")`	faster in loops

Important: always write patterns as **raw strings** with an `r` in front: `r"\d+"`. Otherwise Python may change your backslashes.

Other places regex works

JavaScript:

```
/(\d{4})-(\d{2})/.test("2026-10");              // true
"a1b22".match(/\d+/g);                          // ['1', '22']
"2026-10".replace(/(\d+)-(\d+)/, "$2/$1");      // '10/2026'
```

Command line:

```
grep -E "ERROR|WARN" app.log        # find lines with ERROR or WARN
sed -E 's/[0-9]+/N/g' file.txt      # replace every number with N
```

---

15. Lesson 11: Real-life examples

Check an email

Pattern: `^[\w.+-]+@[\w-]+(?:\.[\w-]+)+$`

Input	Valid?
`a@b.com`	yes
`john.doe+x@mail.co.uk`	yes
`bad@@x`	no
`no_at.com`	no

Check a phone number

Pattern: `^\(?\d{3}\)?[-.\s]?\d{3}[-.\s]?\d{4}$`

Input	Valid?
`555-123-4567`	yes
`(555) 123-4567`	yes
`555.123.4567`	yes
`5551234567`	yes

Input	Valid?
`55-1234`	no

Check a strong password

Pattern: `^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\w\s]).{8,}$`

Each `(?=...)` checks one rule. `.{8,}` checks that there are at least 8 characters.

Input	Valid?	Reason
`Abcdef1!`	yes	all rules met
`abcdef1!`	no	no capital letter
`ABCDEF1!`	no	no small letter
`Abcdefg1`	no	no symbol
`Ab1!`	no	too short

Clean text

Task	Code	Result
Add commas to a number	`re.sub(r"\B(?=(\d{3})+(?!\d))", ",", "1234567")`	`1,234,567`
Remove HTML tags	`re.sub(r"<[^>]+>", "", "<p>Hi <b>there</b></p>")`	`Hi there`
Collapse extra spaces	`re.sub(r"\s{2,}", " ", "a b c")`	`a b c`
Trim spaces from both ends	`re.sub(r"^\s+|\s+$", "", " hi ")`	`hi`

(In the last pattern, write a plain `|` in your code. The backslash is only needed to show it inside this table.)

Take out data

Task	Code	Result
Hashtags	`re.findall(r"#(\w+)", "Love #python and #regex!")`	`['python', 'regex']`

Task	Code	Result
Prices	`re.findall(r"\$\d+(?:\.\d{2})?", "Cost $5, $19.99 or 20")`	`['$5', '$19.99']`

Break a log line into fields

```
log = '192.168.1.10 - ERROR 2026-10-10 took 350ms'
re.search(r"(?P<ip>(?:\d{1,3}\.){3}\d{1,3}).*?(?P<lvl>ERROR|INFO).*?(?
P<ms>\d+)ms", log).groupdict()
# {'ip': '192.168.1.10', 'lvl': 'ERROR', 'ms': '350'}
```

This is exactly what the big project in Section 18 does.

More ready-made patterns

```
Web address:            https?://[^\s/$.?#][^\s]*
Hex colour:             ^#(?:[0-9a-fA-F]{3}){1,2}$
Username (3-16 chars):  ^[a-zA-Z0-9_]{3,16}$
Digits only:            ^\d+$
Decimal number:         ^-?\d+(?:\.\d+)?$
Time, 24 hour:          ^([01]\d|2[0-3]):[0-5]\d$
Date (YYYY-MM-DD):      ^\d{4}-(0[1-9]|1[0-2])-(0[1-9]|[12]\d|3[01])$
IPv4 address:           ^(?:(?:25[0-5]|2[0-4]\d|1?\d?\d)\.){3}(?:25[0-5]|
2[0-4]\d|1?\d?\d)$
```

---

16. Lesson 12: How to build a pattern step by step

Let us build a pattern for dates like `10/05/2026`.

1. **Write an example:** `10/05/2026`
2. **Replace each part with a shortcut:** digits `/` digits `/` digits
3. **Add counts:** `\d{2}/\d{2}/\d{4}`
4. **Allow variations** (`3/7/2026` is also common): `\d{1,2}/\d{1,2}/\d{4}`
5. **Save the parts you need:** `(\d{1,2})/(\d{1,2})/(\d{4})`

6. **Add anchors if you are checking the whole text:** `^(\d{1,2})/(\d{1,2})/(\d{4})$`
7. **Test with good AND bad input.** Note that `13/45/2026` still passes this pattern. Regex checks the **shape**, so check the real ranges in code.

Golden rule: start small, test often, and add one piece at a time.

---

17. Common mistakes

Mistake	What goes wrong	Fix
Using `.*` between quotes	`".*"` on `"a" and "b"` grabs everything	use `".*?"` or `"[^"]*"`
Forgetting `^` and `$`	`\d{3}` also matches inside `abc12345`	add anchors, or use `fullmatch`
Forgetting to escape `.`	`3.14` also matches `3x14`	write `3\.14`
Not using raw strings in Python	`"\d"` and `"\b"` behave strangely	write `r"\d"`
Nested repeats like `(a+)+`	can freeze the program (catastrophic backtracking)	avoid repeating a repeat
Using regex on HTML or JSON	regex cannot handle nested structures	use a proper parser
Over-checking emails	the perfect email pattern is huge	use a simple one, then send a confirmation email
Using user input directly in a pattern	special characters change the meaning	use `re.escape()`

Good habits

1. Test with normal input, **bad input**, empty text, and very long text.
2. Use **named groups** so patterns are readable.
3. Use **verbose mode** (`re.X`) with comments for long patterns.
4. **Compile** patterns that run inside loops.
5. Remember that different languages support slightly different features (this is called a "flavor").

---

18. Big Project: Cleaning messy data

Now we put everything together in a real project.

The problem: a company has messy files (server logs and customer lists). The data has extra spaces, mixed capital letters, different phone and date formats, broken emails, bad IP addresses, and duplicates.

The solution: a small Python program that

1. **reads** the raw files,
2. uses regex to **extract** the fields,
3. uses regex to **clean and check** each field,
4. saves tidy **CSV** files.

The project also has a small web server that accepts **submitted reports** and runs them through the same cleaning steps.

Tools used: Python 3 only (`re` , `csv` , `pathlib` , `http.server`). Nothing extra to install.

Run the batch pipeline:

```
python src/main.py
```

Run the report receiver:

```
cd src
python report_server.py
```

Then, in a second terminal:

```
curl -X POST localhost:8000/submit --data-binary @../data/sample_report.txt
```

19. Project structure: which file does what

```
regex_project/
├── data/
│   ├── raw/                        INPUT: messy, untouched
│   │   ├── server.log
│   │   └── customers.txt
│   ├── cleaned/                    OUTPUT of the batch pipeline
│   │   ├── logs_clean.csv
│   │   └── customers_clean.csv
│   ├── submitted/                  untouched copy of every submitted report
│   ├── reports/
│   │   └── reports_clean.csv       OUTPUT: accepted, cleaned reports
│   └── sample_report.txt           example report for testing
└── src/
    ├── patterns.py
    ├── extractor.py
    ├── cleaner.py
    ├── main.py
    ├── report_parser.py
    └── report_server.py
```

Code files

File	Think of it as	What it does
`main.py`	**The manager**	Starts everything. Calls the reader, then the cleaner, then saves the CSV. Prints a short summary.
`patterns.py`	**The rule book**	Stores every regex in one place: the log pattern, the customer pattern, and the checkers for email, digits, spaces, and dates.
`extractor.py`	**The reader**	Opens a raw file, reads it line by line, and uses regex to turn each line into named fields. Counts lines that do not match.
`cleaner.py`	**The cleaner**	Fixes each field with regex: removes extra spaces, fixes capital letters, standardizes phones and dates, checks emails, rejects bad IPs, removes duplicates.

File	Think of it as	What it does
`report_parser.py`	**The report expert**	Reads one submitted report with a single regex, removes HTML tags from the message, and checks every field. Returns a clean record or a list of errors.
`report_server.py`	**The receiver**	A small web server that listens for `POST /submit`. Saves a raw copy, asks the parser to check the report, avoids duplicates, stores good reports, and replies with JSON.

Data files

File or folder	What it is
`data/raw/server.log`	Sample server log (includes a garbage line, a bad IP, and a duplicate)
`data/raw/customers.txt`	Sample customer records (mixed capitals, spacing, phone and date formats, a broken email, a duplicate)
`data/cleaned/logs_clean.csv`	Cleaned log rows
`data/cleaned/customers_clean.csv`	Cleaned customer rows
`data/submitted/`	Untouched copy of every report received, kept in case you need to reprocess it
`data/reports/reports_clean.csv`	Accepted reports after cleaning
`data/sample_report.txt`	An example report you can submit with `curl`

Which file uses which

```
main.py           uses  extractor.py and cleaner.py
extractor.py      uses  patterns.py
cleaner.py        uses  patterns.py
report_server.py  uses  report_parser.py
report_parser.py  uses  cleaner.py  (clean_name, clean_email, clean_phone,
clean_date)
```

20. Where does raw data come from?

In this project the raw files are **sample data written by hand** for learning. In real life, raw data comes from sources like these:

Source	Example	How it reaches the `raw` folder
Server or app logs	Nginx, Apache, or your own app	The server writes the file automatically, one line per event
Web forms	Signup, contact, or complaint forms	The form sends text to your server, which saves it
Database or CRM exports	Customer lists from a shop or CRM	Someone clicks Export, or a script downloads it on a schedule
Websites (scraping)	Prices, news, product pages	A scraper downloads the page text and saves it
APIs	Payment, weather, or social media services	A script calls the API and saves the answer
Emails	Orders or reports sent by email	A script reads the inbox and saves the text
PDFs and scanned papers	Invoices, forms	Text is read from the file (OCR) and saved
Sensors and devices	Machines, GPS, smart devices	The device sends readings that are stored in files

Why is raw data messy?

Because **different people and different systems write things in different ways**. One person types `(555) 123-4567`, another types `555.987.6543`. Someone uses CAPITAL LETTERS. The same customer shows up twice from two systems.

You cannot fix the source, so you **collect the data as it is** and clean it afterwards:

```
Real source  ->  saved as-is  ->  raw folder  ->  regex extract  ->  regex
clean  ->  cleaned folder
```

21. Practice exercises

Try each one yourself before looking at the answers.

#	Task	Text to test
1	Find all hashtags (without the `#`)	`Love #python and #regex!`
2	Find all prices	`Cost $5, $19.99 or 20`
3	Remove all HTML tags	`<p>Hi <b>there</b></p>`
4	Add commas to a number	`1234567`
5	Collapse repeated spaces into one	`a  b  c`
6	Find numbers that are followed by `%`	`50% 20 30%`
7	Check if a text is made of digits only	`123` and `12a`

Answers

#	Pattern	Expected result
1	`#(\w+)`	`python`, `regex`
2	`\$\d+(?:\.\d{2})?`	`$5`, `$19.99`
3	`<[^>]+>` (replace with nothing)	`Hi there`
4	`\B(?=(\d{3})+(?!\d))` (replace with `,`)	`1,234,567`
5	`\s{2,}` (replace with one space)	`a b c`
6	`\d+(?=%)`	`50`, `30`
7	`^\d+$`	`123` passes, `12a` fails

Mini projects to try

1. Take a text file of names and phone numbers. Write one pattern to extract both.
2. Take a copy of any chat or email. Find every web link in it.
3. Take a spreadsheet column of dates in mixed formats. Convert them all to `YYYY-MM-DD`.

22. Cheat sheet

Characters

Symbol	Meaning
`.`	any one character
`\d` / `\D`	digit / not a digit
`\w` / `\W`	word character / not a word character
`\s` / `\S`	whitespace / not whitespace
`[abc]`	a, b, or c
`[a-z]`	any letter from a to z
`[^abc]`	anything except a, b, c
`\.` `\$` `\(`	the real symbol (escaped)

How many

Symbol	Meaning
`*`	0 or more
`+`	1 or more
`?`	0 or 1 (optional)
`{n}`	exactly n
`{n,m}`	between n and m
`*?` `+?`	lazy: as few as possible

Position

Symbol	Meaning
`^`	start
`$`	end
`\b`	word edge

Groups and choices

Symbol	Meaning
`( )`	capture group
`(?: )`	group without saving
`(?P<name> )`	named group (Python)
`\1`	reuse group 1
`a|b`	a OR b

Peeking

Symbol	Meaning
`(?=...)`	followed by
`(?!...)`	not followed by
`(?<=...)`	preceded by
`(?<!...)`	not preceded by

Python functions

`search` `match` `fullmatch` `findall` `finditer` `sub` `split` `compile` `escape`

---

23. Small dictionary of words

Word	Simple meaning
Pattern	the regex you write, describing a shape
Match	a piece of text that fits the pattern
Regex engine	the program that runs your pattern on text
Character class	a set of allowed characters, like `[aeiou]`
Quantifier	a symbol that says how many times, like `+` or `{3}`
Greedy	takes as much text as it can
Lazy	takes as little text as it can

Word	Simple meaning
Anchor	matches a position (start, end, word edge), not a letter
Group	part of a pattern wrapped in `( )`
Capture	saving the text matched by a group
Backreference	reusing saved text inside the pattern, like `\1`
Escape	putting `\` before a special symbol so it means itself
Lookaround	peeking at nearby text without including it
Flag	a setting that changes how the pattern behaves
Backtracking	the engine going back to try another way
Raw string	a Python string written as `r"..."`, so backslashes stay as they are
Extract	pull useful pieces out of messy text
Clean	fix text so every record follows the same format
Validate	check that data follows the rules
Duplicate	the same record appearing more than once
Raw data	data exactly as it arrived, not yet cleaned
CSV	a simple table saved as text, with commas between values
Log file	a file where a program writes down what happened
Pipeline	a chain of steps: read, extract, clean, save

---

24. What to do next

1. Open **regex101.com**. Paste any pattern from this guide. It explains every part in plain words.
2. Redo the **practice exercises** without looking at the answers.
3. Open `patterns.py` in the project. You can now read every pattern in it.
4. Change the project to use **your own data**: a different log format, a contact list, or a spreadsheet export.
5. Come back to this guide whenever you forget something. Use the **cheat sheet** in Section 22.

Final advice: nobody memorizes regex. Everyone builds patterns **one small piece at a time and tests often**. If you can do that, you can use regex.